Grothendieck's Problem-Solving Process

Three principles for AI Engineering

Dr. Ye Kyaw Thu

August 15, 2026

Lab Leader, Language Understanding Lab, AI Mario, Myanmar

Outline

Why Grothendieck, why now?

A short biography

What he actually did, in plain words

The three principles

Principle 1: Let the problem sit

Principle 2: Abstraction

Principle 3: Sharpen the axe

Synthesis

Why Grothendieck, why now?

Why should an AI engineer care?

Alexander Grothendieck (1928–2014) rebuilt algebraic geometry from the ground up.

- Wrote $\sim$10,000 pages of mathematics.
- Fields Medal (1966), refused to attend.
- His *way of thinking* is what we will steal.

Thesis of this talk: the same mental discipline that lets a mathematician see the right generality is exactly what lets an AI engineer build models that generalize.

Why should an AI engineer care?

AI Engineering parallels

Let it sit	LR cooldown, EMA (Exponential Moving Average), annealing
Abstraction	Embeddings, foundation models
Sharpen axe	Pre-train, then fine-tune

A short biography

Who was Alexander Grothendieck?

Alexander Grothendieck in Montréal (1970)

The IHES Seminars

The SAG (Séminaire de Géométrie Algébrique) seminar at the IHES (Institut des hautes études scientifiques)

A life in one timeline

- Born Berlin 1928, father Sascha Schapiro (anarchist, Jewish, deported to Auschwitz 1942).
- Hidden in France during WWII, school in Le Chambon-sur-Lignon.
- PhD with Laurent Schwartz (Nancy), nuclear spaces.
- IHES 1958–1970: the EGA / SGA revolution.
- Resigned over military funding; isolated life 1991–2014.

Childhood: shaped by absence and survival

- Father: Alexander Schapiro, Ukrainian-Jewish anarchist, lost an arm escaping Tsarist prison, deported from Le Vernet to Auschwitz (1942).
- Mother: Hanka Grothendieck, bohemian writer, anti-bourgeois.
- 1939–1945: internment camps, hidden children, the village of *Le Chambon* that saved Jewish children.

A housemother's note, 1943

"Alexandre Grothendieck, dit Alex le Poète…très intelligent, toujours plongé dans ses réflexions, ses lectures…réclame le silence pour écouter la musique."

Always deep in thought; demanded silence to listen to music. A pattern set early.

The IHES decade (1958–1970)

- Joined the Institut des Hautes Études Scientifiques near Paris.
- Ran legendary seminars: EGA (Éléments de géométrie algébrique) and SGA (Séminaire de géométrie algébrique).
- Surrounding himself with co-authors (Dieudonné, Serre, Artin, Deligne, Verdier…).
- 1966: Fields Medal awarded in Moscow; he refused to travel for political reasons.
- 1970: discovered IHES accepted (minor) military funding; resigned.

Engineering lesson

Grothendieck's *output* was not solitary genius — it was a well-designed research program. Like a great engineering org: clear vision, division of labor, relentless documentation.

After 1970: the long silence

- Founding of *Survivre et vivre* — ecological / pacifist movement.
- Visited Hanoi (1967) to denounce the Vietnam war.
- Professor at Montpellier, then CNRS (1984).
- 1991: burns papers, disappears to the Pyrenees.
- Writes thousands of unpublished pages: *Récoltes et Semailles*, *La Clef des Songes*, *Les Dérivateurs*.
- Dies 13 November 2014 in Saint-Lizier.

From Récoltes et Semailles (Harvesting and Sowing)

"...cette harmonie...qui unfurl au cours des générations et des siècles, par le contrepoint délicat de tous les thèmes apparaissant à tour de rôle, comme si appelés du vide."

Translation: From *Récoltes et Semailles*

English translation

"And every science, when we understand it not as an instrument of power and domination but as an adventure in knowledge pursued by our species across the ages, is nothing but this harmony, more or less vast, more or less rich from one epoch to another, which unfurls over the course of generations and centuries, by the delicate counterpoint of all the themes appearing in turn, as if summoned from the void."

— *Alexander Grothendieck*

Translation: From *Récoltes et Semailles*

Myanmar translation

“လူသားမျိုးနွယ်၏ သိပ္ပံပညာသည် တစ်ယောက်တည်း တစ်ခါတည်း ဖန်တီးထားခြင်း မဟုတ်ပါ။ ရာစုနှစ်များစွာ ကြာသွားသည်နှင့် အမျှ၊ မတူညီသော သိပ္ပံအယူအဆ များသည် တစ်ခုပြီးတစ်ခု ထွက်ပေါ်လာကြသည်။ ထိုအယူအဆများသည် အချင်းချင်း မည်သည့်အခါမှ ပဋိပက္ခမဖြစ်ပဲ၊ ဂီတတွင် တေးသွားများ အပြန်အလှန်ဖြည့်စွက်သကဲ့သို့ အလွန်နူးညံ့သိမ်မွေ့စွာ အပြန်အလှန် ထောက်ပံ့ဖြည့်စွက်ကြခြင်းဖြင့်သာ ယနေ့ ကမ္ဘာကြီးပေါ်ရှိ အဆင့်မြင့် သိပ္ပံပညာရပ်များကို ဘာမှမရှိသော ဗလာနတ္ထိမှ တဖြည်းဖြည်း ဖိတ်ခေါ်ဆောင်ကြဉ်းလာသကဲ့သို့ ဖန်တီးပေးနိုင်ခြင်းဖြစ်သည်။”

— *Alexander Grothendieck*

What he actually did, in plain words

Five big contributions — conceptually

1. **Tohoku paper (1957).** Re-derived homological algebra from scratch (didn't wait for Cartan–Eilenberg's manuscript). Introduced *abelian categories*.
2. **Schemes (1958).** Replaced "varieties" by a much more general object so that intersection, base change, reduction modulo p all become natural.
3. **Grothendieck topology / étale cohomology.** Invented a "topology" fine enough to host a Weil cohomology.
4. **Riemann–Roch theorem (with Borel, Serre).** Generalized from curves to arbitrary morphisms; introduced K-theory.
5. **Standard conjectures (1969).** Still open today — Deligne proved Weil (1973) by a different route.

The nut and the hammer

Grothendieck, *Récoltes et Semailles*

"...the first analogy that came to my mind is of immersing the nut in some softening liquid, and why not simply water? From time to time you rub so the liquid penetrates better; otherwise you let time pass. The shell becomes more flexible through weeks and months — when the time is ripe, hand pressure is enough, the shell opens like a perfectly ripened avocado!"

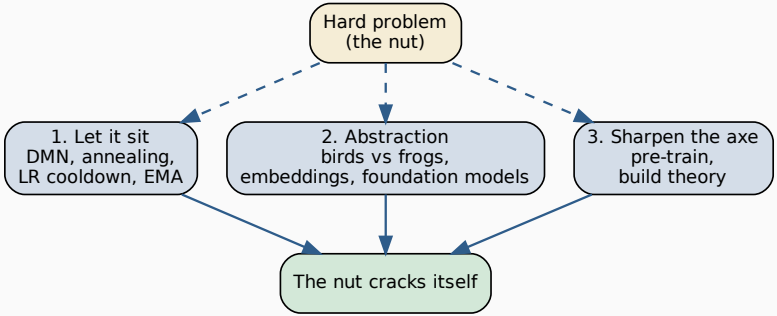
Two strategies:

- Hammer the nut (problem-solver).
- Soften the nut (theory-builder).

Grothendieck chose the second. The three principles that follow are different facets of *softening*.

The three principles

Three principles, one diagram



- Each principle is a *discipline*, not a trick.
- In AI engineering they correspond to specific practices we will make concrete with code.

Principle 1: Let the problem sit

1. Let the problem sit

Grothendieck

"From time to time you rub so the liquid penetrates better; otherwise, you let time pass."

What is happening cognitively?

- Activating the Default Mode Network (Raichle et al., 2001): mind-wandering, introspection, creativity.
- Insights surface when you stop forcing them.
- "Incubation effect" in creative cognition (Wallas 1926, Sio & Ormerod 2009).

For AI Engineering

In optimization: when the loss plateaus, you do *not* throw bigger gradients at it. You reduce the learning rate, you let the model "settle" — this is the algorithmic incarnation of the same idea.

Where “letting it sit” appears in ML

Training-time analogues

- Learning-rate cooldown (cosine, one-cycle).
- Early stopping with *patience*.
- Exponential Moving Average of weights (a “settled” model).
- Stochastic Weight Averaging.

Inference-time analogues

- Chain-of-Thought — let the model “think”.
- Test-time training / adaptation.
- Simulated annealing’s cooling schedule.
- Diffusion sampling: “denoise gradually”.

Takeaway

A nut you cannot crack today may crack tomorrow if the surrounding water has been doing its work. Schedule time for the model — and for yourself — to settle.

Code: a trainer that “lets the loss sit”

```python
import torch
from copy import deepcopy

class PatienceCooler:
    """When loss stalls, soften the optimizer (drop LR) instead of hammering.
       """
    def __init__(self, patience=3, factor=0.5, min_lr=1e-7):
        self.patience, self.factor, self.min_lr = patience, factor, min_lr
        self.best, self.bad = float('inf'), 0

    def step(self, val_loss, optimizer):
        if val_loss < self.best - 1e-4:
            self.best, self.bad = val_loss, 0
        else:
            self.bad += 1
            if self.bad >= self.patience:
                for g in optimizer.param_groups:
                    g['lr'] = max(g['lr']*self.factor, self.min_lr)
                self.bad = 0  # let the model sit again

```

Code: simulated annealing — the algorithm that *is* "letting it sit"

```python
import numpy as np

def simulated_annealing(f, x0, T0=1.0, T_min=1e-4, alpha=0.99, steps=20, seed=0):
    """The temperature schedule IS Grothendieck's water:
    hot at first (rubbing), then cool (letting it sit)."""
    rng = np.random.default_rng(seed)
    x = np.asarray(x0, dtype=float)
    fx, T = f(x), T0
    best_x, best_fx = x.copy(), fx
    while T > T_min:
        for _ in range(steps):
            x_new = x + rng.normal(scale=T, size=x.shape)   # jostle
            f_new = f(x_new)
            dF = f_new - fx
            if dF < 0 or rng.random() < np.exp(-dF / T):
                x, fx = x_new, f_new                          # accept
                if fx < best_fx: best_x, best_fx = x.copy(), fx
        T *= alpha                                            # cool: let it sit
    return best_x, best_fx
```

NLP example: chain-of-thought as “letting it sit”

Direct prompt:

```
Q: A bat and ball cost $1.10.
   The bat costs $1.00 more
   than the ball. How much
   is the ball?

A: $0.10              (wrong)
```

Chain-of-thought (letting the model “sit” with the problem):

```
Q: (same)

A: Let x = ball price.
   Bat = x + 1.00.
   2x + 1.00 = 1.10
   x = 0.05            (correct)
```

Takeaway

Giving the model more tokens to “think” = letting the problem sit. Reasoning is a cooling schedule over the latent space.

Principle 2: Abstraction

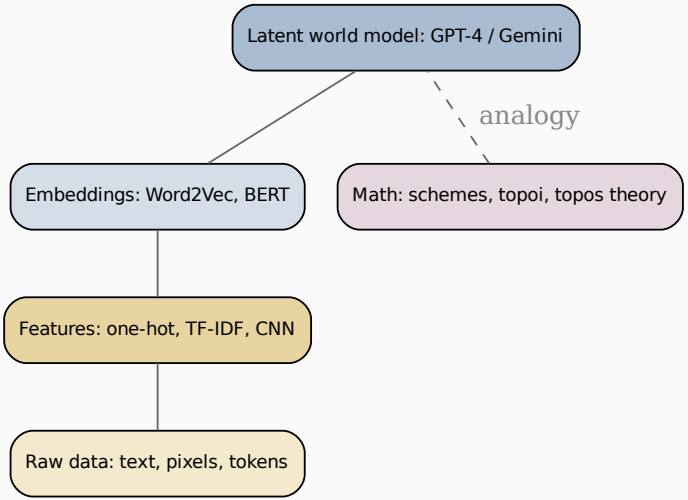
2. Abstraction — the bird's-eye view

Freeman Dyson, 2008

"Some mathematicians are birds, others are frogs. Birds fly high in the air and survey broad vistas of mathematics out to the far horizon...Frogs live in the mud below and see only flowers that grow nearby."

- To Grothendieck, a problem was *not solved* until viewed from the right general perspective.
- He didn't solve one problem at a time — he solved many at once by lifting them to a common framework.
- Hyman Bass: "an extremely powerful, almost other-worldly ability of abstraction."

Abstraction pyramid (math $\leftrightarrow$ ML)



Code: an autoencoder is *abstraction in code*

```python
import torch.nn as nn

class Autoencoder(nn.Module):
    """The latent z is the 'right general perspective':
    a low-dimensional abstraction from which the data is reconstructable."""
    def __init__(self, in_dim=784, hidden=256, latent=32):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(in_dim, hidden), nn.GELU(),
            nn.Linear(hidden, latent))
        self.decoder = nn.Sequential(
            nn.Linear(latent, hidden), nn.GELU(),
            nn.Linear(hidden, in_dim))
    def encode(self, x):   # raw → abstraction
        return self.encoder(x)
    def forward(self, x):
        z = self.encode(x)
        return self.decoder(z), z
```

Code: an autoencoder is *abstraction in code*

The Grothendieck move

Don't ask "how do I classify this image?" Ask: what is the minimal latent that re-generates it? The classifier then becomes a small surface phenomenon on top of the abstraction.

Code: contrastive abstraction (SimCLR-style)

```python
import torch, torch.nn as nn, torch.nn.functional as F

class ContrastiveAbstraction(nn.Module):
    """Two views of the same image should map to the same point in z.
    Forces discarding of augmentation-specific detail."""
    def __init__(self, backbone, proj_dim=128):
        super().__init__()
        self.backbone = backbone
        self.proj = nn.Linear(backbone.dim, proj_dim)
    def forward(self, x1, x2):
        z1 = F.normalize(self.proj(self.backbone(x1)), dim=1)
        z2 = F.normalize(self.proj(self.backbone(x2)), dim=1)
        logits = z1 @ z2.T / 0.5  # NT-Xent loss
        labels = torch.arange(z1.size(0), device=z1.device)
        return F.cross_entropy(logits, labels)
```

Code: contrastive abstraction (SimCLR-style)

What this teaches

"What stays invariant under all the changes I can imagine?" — that invariant is the abstraction. Grothendieck asked the same question about algebraic varieties under base change.

Real-world NLP: from one-hot to meaning

Representation	Dim	What it captures
One-hot	$\lvert V \rvert$ (e.g. 50k)	Identity only — no relations
Word2Vec	100–300	Linear analogies: $\vec{king} - \vec{man} + \vec{woman} \approx \vec{queen}$
BERT embedding	768	Contextual, sense-disambiguated
GPT-4 latent	$\sim$ 12288	World-model, instruction-following

Abstraction as compression-with-meaning

Each layer lifts the representation to a more general perspective from which more tasks become trivial. This is Grothendieck's move from varieties $\to$ schemes $\to$ topoi.

A live demo: does abstraction survive noise?

Question. Two unsupervised models were trained on 1,797 handwritten digit images (8×8 pixels, no labels shown to either model). Which representation makes the digit easiest to recover later — especially once the input is corrupted?

- **Raw pixels** — no abstraction, the baseline.
- **Autoencoder latent** — trained only to *reconstruct* (the code from the last slide set).
- **Contrastive latent** — trained only so that noisy/shifted copies of the *same* image map to the same point (SimCLR-style).

Why this tests Grothendieck's idea directly

Neither model was ever told which digit is which. If a representation still makes the digit easy to recover, that structure was discovered, not supervised — exactly "what stays invariant."

Experimental setup

Model	Trained on	Dims
Raw pixels	— (baseline)	64
Autoencoder	reconstruction loss, 200 epochs	8
Contrastive (SimCLR)	NT-Xent loss, 800 epochs	32

The corruption used at test time (never seen during evaluation of raw pixels, only during contrastive *training*):

- Gaussian pixel noise.
- Random pixel dropout (15%).
- Small random spatial shift (± 1 px).

Experimental setup

The actual test

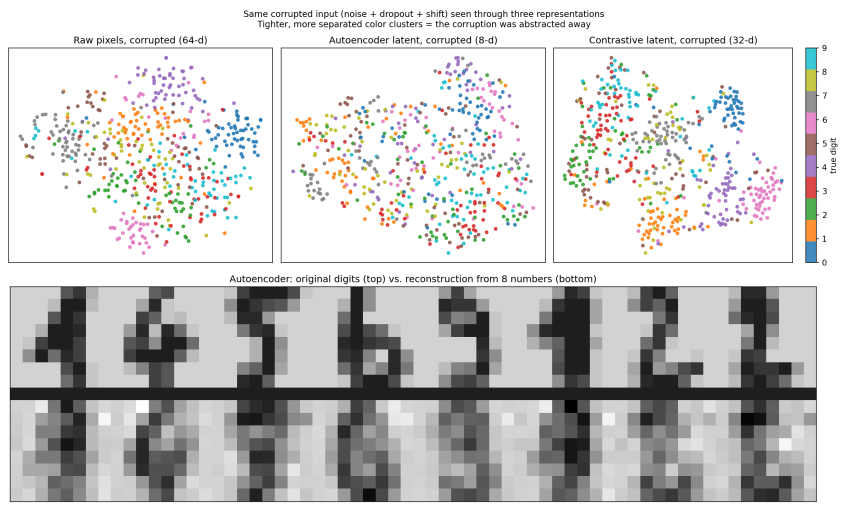
A k-NN classifier is fit on clean training representations, then scored on corrupted test representations — five random corruptions, averaged. No representation is retrained for this.

Result: accuracy, clean vs. corrupted

Representation	Dims	Clean acc.	Corrupted acc.
Raw pixels	64	0.980	0.449 ± 0.237
Autoencoder latent	8	0.870	0.454 ± 0.151
Contrastive latent	32	0.883	0.501 ± 0.063

- On **clean** images raw pixels look best — there is nothing to abstract away yet, so abstraction cannot help.
- On **corrupted** images the contrastive latent wins on *both* axes: highest mean accuracy, and by far the smallest $\pm$ — it barely reacts to which random corruption was drawn.

Result: what the representations look like



Understanding the result

Grothendieck's question, made measurable

"What stays invariant under all the transformations I'm willing to apply?"

- The stability gap (± 0.237 vs. ± 0.063) matters more than the mean: raw pixels change answer almost at random once corrupted; the contrastive latent was *built* to resist exactly that kind of change.
- The autoencoder helps a little (denoising is a side-effect of compressing to 8 numbers) but was never asked to be invariant to noise — only to reconstruct clean input. It compresses; it does not specifically discard nuisance detail.
- The contrastive model was given no labels either — only the instruction "these two corrupted versions are the same object." That one instruction was enough to discover the digit structure on its own.

Back to the nut

Grothendieck softened the nut by finding the right general perspective. Here, softening the nut meant: don't ask a network to memorize pixels — ask it what should stay the same.

Principle 3: Sharpen the axe

3. Sharpening the axe

Abraham Lincoln (apocryphal)

"Give me six hours to chop down a tree and I will spend the first four sharpening the axe."

Grothendieck

"He wanted to build the theory in which his problems would lie naturally, instead of having to force it."

The AI Engineering translation

Pre-training is sharpening the axe. Fine-tuning is chopping the tree. The *vast reservoir of knowledge* Grothendieck built — analogous to a model that has seen a vast reservoir of data — is what makes the downstream task easy.

The pre-train → fine-tune paradigm

- **Phase 1 (sharpen).** Train on a huge, generic corpus with a self-supervised objective. No specific task.
- **Phase 2 (chop).** Adapt to your specific task with a small amount of labeled data and a few gradient steps.
- The model has built a theory (latent world model) on which the problem "lies naturally."

Code: the paradigm in 25 lines

```python
import torch, torch.nn as nn
from torch.optim import AdamW

class TinyLM(nn.Module):
    """A toy language model. Phase 1 sharpens it; Phase 2 chops."""
    def __init__(self, vocab=10000, dim=256, n_layers=4):
        super().__init__()
        self.dim = dim
        self.tok = nn.Embedding(vocab, dim)
        self.tr  = nn.TransformerEncoder(
            nn.TransformerEncoderLayer(dim, nhead=4, batch_first=True),
            num_layers=n_layers)
        self.head = nn.Linear(dim, vocab)
    def forward(self, x):  return self.head(self.tr(self.tok(x)))
```

Code: the paradigm in 25 lines

```
def pretrain(model, dl, epochs=3, lr=1e-4):        # 1. SHARPEN
    opt = AdamW(model.parameters(), lr=lr)
    for _ in range(epochs):
        for x, y in dl:
            loss = nn.functional.cross_entropy(model(x), y)
            opt.zero_grad(); loss.backward(); opt.step()
    return model
```

Code: the paradigm in 25 lines

```
def finetune(model, tdl, n_classes=2, epochs=1, lr=1e-5): # 2. CHOP
    model.head = nn.Linear(model.dim, n_classes)
    opt = AdamW(model.parameters(), lr=lr)
    for _ in range(epochs):
        for x, y in tdl:
            logits = model(x)[:, 0, :]  # CLS-token style
            loss = nn.functional.cross_entropy(logits, y)
            opt.zero_grad(); loss.backward(); opt.step()
    return model
```

Numbers: why sharpening pays

Setting	Pre-train compute	Downstream data	Downstream acc.
From scratch (GLUE)	—	550k examples / task	70–80%
$\text{BERT}_{\text{Base}}$ fine-tune	$\sim 10^{20}$ FLOPs	a few thousand	84–88%
GPT-3 zero-shot	$\sim 3 \cdot 10^{23}$ FLOPs	0 (prompt only)	50–75%
GPT-4 few-shot	$\sim 2 \cdot 10^{25}$ FLOPs	5–10 examples	85–90%

Pattern, exactly as Grothendieck predicted

The more you invest in the general theory (pre-training), the less effort each specific problem (downstream task) requires — sometimes to zero.

The cost side: when NOT to sharpen

- If you only have *one* tree to chop, the axe-sharpening is not worth it.
- Pre-training pays off when the abstraction is reused many times — Grothendieck's EGA was reused by thousands.
- For one-off tasks, fine-tune an existing foundation model rather than pre-train your own.

Engineering rule of thumb

Sharpen the axe when:

$$(\text{cost of pre-train}) < \sum_{\text{tasks } i} (\text{cost saved on task } i)$$

This is the engineer's version of Grothendieck's bet on generality.

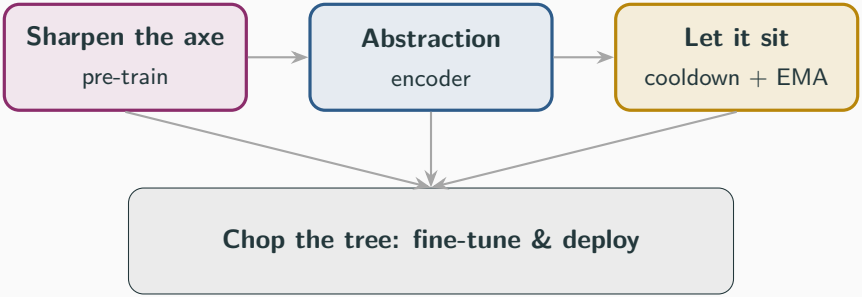
Grothendieck's bet on generality

The ROI (Return on Investment) of EGA & SGA

Grothendieck spent 12 years at IHES "sharpening the axe" by writing EGA and SGA (thousands of pages of abstract theory). That was a massive upfront cost of his time. But he did it because he knew it would allow future generations of mathematicians to solve thousands of individual problems without having to reinvent the wheel every time. His "pre-training" paid off because the downstream savings to mathematics were immense.

Synthesis

The three principles, working together



Engineering reading

Build a strong, abstract foundation (1+2). Let the training dynamics settle (3). The downstream task then *falls out by itself* — like a ripe mango.

Checklist for your next ML project

1. **Before coding:** spend a day writing down what is *essential* vs *accidental* in your problem. (Abstraction.)
2. **Before training:** check whether a pre-trained foundation exists for your modality. If yes, fine-tune. If no, plan pre-training. (Sharpen the axe.)
3. **During training:** set a patience-based LR schedule. Track EMA weights. Don't panic at plateaus. (Let it sit.)
4. **When stuck:** walk away. Read a related paper. Sketch the problem at a higher level of generality. Come back. (All three.)
5. **After shipping:** write up what you learned in a form reusable for the *next* problem. (Building theory for your future self.)

A personal coda

Grothendieck, *Récoltes et Semailles*

"...every science, when we understand it not as an instrument of power and domination but as an adventure in knowledge...is nothing but this harmony...as if summoned from the void."

- The three principles are not really about mathematics, nor really about ML.
- They are about patience with structure.
- That patience is what distinguishes a *craft* from a *hack*.

Thank you.

Questions, counter-examples, abstractions?

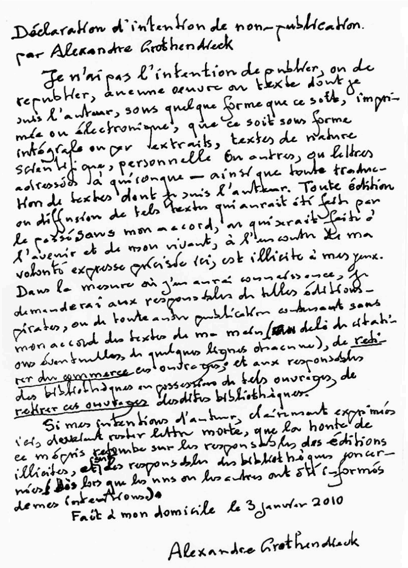
Further reading & sources

- C.-L. Chai & F. Oort, *Life and work of Alexander Grothendieck*. `https://www.math.upenn.edu/~chai/`
- A. Jackson, *As if summoned from the void*, Notices AMS (2004).
- W. Scharlau, *Wer ist Alexander Grothendieck?* (2007–2010).
- A. Grothendieck, *Récoltes et Semailles* (1985).
- C. Olah, *Neural Networks, Types, and Functional Programming* — a programmer's view of abstraction in NNs.
- Sio & Ormerod, *Does incubation work?* Psychonomic Bulletin (2009).
- **Data sources for the "Sharpening pays" table:**
- Wang et al., *GLUE: A Multi-Task Benchmark and Analysis Platform* (2018).
- Devlin et al., *BERT: Pre-training of Deep Bidirectional Transformers* (2018).
- Brown et al., *Language Models are Few-Shot Learners (GPT-3)* (2020).
- OpenAI, *GPT-4 Technical Report* (2023); compute estimated via Epoch AI.

Resources: EGA, SGA & Translations

- Community translation (French to English) of EGA (Désolé, Grothendieck.): github.com/ryankeleti/ega
- An English Translation of Grothendieck's SGA: github.com/jmoellermath/translate-SGAI
- Grothendieck's letter (Secret Blogging Seminar): sbseminar.wordpress.com/...
- SGA Brought to you by the Bourbakistas (SLMath): library.slmath.org/nonmsri/sga
- SGA pdf files by Antoine Chambert-Loir: library.slmath.org/nonmsri/sga/sga/pdf
- Algebraic Geometry and Arithmetic Curves (Qing Liu): math.u-bordeaux.fr/q̃liu/Book
- Séminaire de Géométrie Algébrique du Bois Marie: github.com/thosgood/sga

Declaration of Intent (Original Handwritten)


Déclaration d'intention de non-publication.
par Alexandre Grothendieck

Je n'ai pas l'intention de publier, ou de republier, aucune oeuvre ou texte dont je suis l'auteur, sous quelque forme que ce soit, imprimée ou électronique, que ce soit sous forme intégrale ou par extraits, textes de nature scientifique, personnelle ou autres, ou lettres adressées à quiconque – ainsi que toute traduction de textes dont je suis l'auteur. Toute édition ou diffusion de tels textes qui aurait été faite par le passé sans mon accord, ou qui serait faite à l'avenir et de mon vivant, à l'encontre de ma volonté expresse précisée ici, est illicite à mes yeux.
Dans la mesure où j'en aurai connaissance, je demanderai aux responsables de telles éditions pirates, ou de toute autre publication contenant sans mon accord des textes de ma main (au delà de citations de quelques lignes chacune), de retirer du commerce ces ouvrages; et aux responsables des bibliothèques en possession de tels ouvrages, de retirer ces ouvrages desdites bibliothèques.
Si mes intentions d'auteur, clairement exprimées ici, devaient rester lettre morte, que la honte de ce mépris retombe sur les responsables des éditions illicites, et sur les responsables des bibliothèques concernés (dès lors que les uns ou les autres ont été informés de mes intentions).
Fait à mon domicile le 3 janvier 2010

Alexandre Grothendieck

Source & Context:

In early 2010, Grothendieck sent this declaration to a friend, explicitly requesting that all his unpublished works, letters, and translations be removed from commerce and libraries.

Link to the letter:

https://sbseminar.wordpress.com/2010/02/09/grothendiecks-letter/

Declaration of Intent of Non-Publication (English)

January 3, 2010

I do not intend to publish or republish any work or text of which I am the author, in any form whatsoever, printed or electronic, whether in full or in excerpts, texts of personal nature, of scientific character, or otherwise, or letters addressed to anybody, and any translation of texts of which I am the author. Any edition or dissemination of such texts which have been made in the past without my consent, or which will be made in the future and as long as I live, is against my will expressly specified here and is unlawful in my eyes. As I learn of these, I will ask the person responsible for such pirated editions, or of any other publication containing without permission texts from my hand (beyond possible citations of a few lines each), to remove from commerce these books; and librarians holding such books to remove these books from those libraries.

If my intentions, clearly expressed here, should go unheeded, then the shame of it falls on those responsible for the illegal editions, and those responsible for the libraries concerned (as soon as they have been informed of my intention).

Written at my home, January 3, 2010,
Alexandre Grothendieck.